

Teste de Software

Roteiro *Sprint 2* - Property-based testing com Hypothesis

Objetivo: até aqui o GitHub Copilot gerou testes baseados em exemplos a partir da sua Q8 (Sprint 1) e a fixture de isolamento a partir da sua observação crítica (*Sprint 2*). Neste *sprint*, a ferramenta vai gerar testes baseados em propriedades a partir de invariantes do domínio bancário que você mesmo formulará em linguagem natural. O salto é epistemológico: deixar de pensar em casos pontuais e passar a pensar em propriedades universais que o sistema precisa respeitar para qualquer entrada possível, sustentando a análise em MacIver et al. (2019).

Conceito essencial: testes baseados em exemplos verificam casos específicos fixos - “transferir 100 com saldo 1000 deve resultar em saldo 900”. Testes baseados em propriedades verificam invariantes - “para qualquer valor positivo menor ou igual ao saldo, a transferência deve ser aceita e a soma total dos saldos deve ser conservada”. A biblioteca Hypothesis gera centenas de inputs automaticamente e, ao encontrar um contraexemplo que viola a propriedade, faz *shrinking* - reduz o contraexemplo ao menor caso possível que ainda quebra a propriedade, facilitando o diagnóstico (MacIver et al., 2019, p. 2).

Passo 1. Em seu arquivo Word, formule **três invariantes** do domínio do sistema de internet banking. Cada invariante deve ter a forma “para qualquer *X* em domínio *D*, a propriedade *P* se verifica”. A primeira está formulada abaixo como referência; formule a **segunda** e a **terceira** você mesmo, com duas pistas:

Invariante 1 (referência): para qualquer valor não-positivo (zero ou negativo) submetido ao endpoint /transferencia, o sistema deve sempre retornar HTTP 422 - valor inválido.

Pista para a invariante 2: revise a coluna Missing do relatório de cobertura registrado no Passo 7 do Sprint 2: quais cenários do endpoint /saldo nunca foram ativados pela suíte gerada pela IA? Qual invariante universal eles representam?

Pista para a invariante 3: em qualquer sistema bancário, uma transferência aceita deve preservar uma propriedade matemática fundamental entre as duas contas envolvidas. Qual é essa propriedade - e como ela se expressa em termos de saldos antes e depois da operação?

Passo 2. Verifique três dependências antes de avançar:

- (a) o arquivo `confstest.py` produzido no Sprint 2 deve estar dentro da pasta `internet_banking_tests` - a fixture `autouse` é o que garantirá ao Hypothesis um banco em estado conhecido antes de cada um dos centenas de runs;
- (b) o servidor Flask deve estar rodando no Terminal 1 com `python app.py`;
- (c) a biblioteca `hypothesis` deve estar instalada na `venv` (já consta no `pip install` do pré-requisito; confirme no Terminal 2 com `pip show hypothesis`).

Sprint 2 – passo 2 - modelo de três invariantes – não copie – pense...

Referência didática: as três invariantes abaixo são exemplos formulados pelo professor para você usar como modelo ao adaptar o prompt do Passo 3. Use-as como base de comparação com as invariantes que você formulou por conta própria - não como cópia direta. Quanto mais próxima da sua a invariante final, mais consistente foi seu raciocínio sobre o domínio bancário.

Invariante 1 - Rejeição de valor não-positivo no `/transferencia`

Para qualquer valor não-positivo (zero ou negativo) submetido ao endpoint `/transferencia`, o sistema deve sempre retornar HTTP 422 — valor inválido.

Invariante 2 - Resposta universal a conta inexistente no `/saldo`

Raciocínio: no Sprint 1 a IA gerou um único teste para conta inexistente - algo como `test_saldo_conta_inexistente` usando um único ID específico (por exemplo, 99). Isso é um exemplo pontual: comprova que para aquele ID o retorno é 404. Mas não comprova que *todo* ID fora do conjunto cadastrado retorna 404 - o sistema poderia, em tese, ter um caminho de código defeituoso para um ID específico (ex.: ID 7, ID 100, ID 9999) e a suíte da IA nunca descobriria. A invariante universal afirma que o comportamento se repete para *qualquer* ID inteiro fora dos IDs cadastrados `{1, 2, 3}`.

Para qualquer ID inteiro que não pertença ao conjunto de contas cadastradas no sistema (ou seja, $ID \notin \{1, 2, 3\}$), o endpoint `/saldo/<conta_id>` deve sempre retornar HTTP 404 - conta não encontrada.

Invariante 3 - Conservação de valor em transferência aceita

Raciocínio: uma transferência bancária é, matematicamente, uma redistribuição: o que sai de uma conta entra na outra. Nada se cria, nada se perde. Se Alice tinha R\$ 1000 e Bob tinha R\$ 500, a soma é R\$ 1500. Após uma transferência de R\$ 300 entre elas (aceita pelo sistema), a soma deve continuar sendo exatamente R\$ 1500 - agora distribuída como R\$ 700 e R\$ 800. Esse é o **princípio de conservação de valor** - uma das invariantes mais críticas em qualquer sistema financeiro, porque sua violação significaria criação ou destruição de dinheiro por defeito de software. A IA do Sprint 1 testou *casos pontuais* de transferência com valores específicos, mas não verificou esta invariante universal - para qualquer valor válido, ela deve se manter.

Para qualquer transferência aceita pelo endpoint `/transferencia` (resposta HTTP 200), a soma dos saldos das contas de origem e destino imediatamente após a operação deve ser exatamente igual à soma dos saldos imediatamente antes da operação — $\text{saldo}(\text{origem}, \text{antes}) + \text{saldo}(\text{destino}, \text{antes}) = \text{saldo}(\text{origem}, \text{depois}) + \text{saldo}(\text{destino}, \text{depois})$.

Passo 3. Abra o GitHub Copilot Chat no VSCode (Cmd+Shift+I no macOS / Ctrl+Alt+I no Windows). No editor, crie um novo arquivo chamado `test_hypothesis.py`. **COLE O PROMPT ABAIXO** no campo de chat do GitHub Copilot, substituindo `[SUAS_TRES_INVARIANTES]` pelo texto das três invariantes que você formulou no Passo 1, e pressione Enter:

Gere o conteúdo de um arquivo `test_hypothesis.py` para testes baseados em propriedades com a biblioteca Hypothesis em Python, aplicados ao sistema de internet banking da disciplina de Teste de Software.

Servidor Flask local em `http://127.0.0.1:5000` com os seguintes endpoints:

- GET `/saldo/<conta_id>`
- POST `/transferencia` — corpo JSON: `{"origem":int,"destino":int,"valor":float}`
- GET `/extrato/<conta_id>`

Estado inicial das contas (mantido pelo conftest.py existente na mesma pasta, com fixture autouse): id=1 saldo 1000.00; id=2 saldo 500.00; id=3 saldo 0.00.

Invariantes a serem implementadas como propriedades:

[SUAS_TRES_INVARIANTES]

Requisitos obrigatórios:

- Use `@given` e estratégias `strategies` as `st` da biblioteca `hypothesis` para gerar inputs automaticamente.
- Use `assume(...)` quando a propriedade só fizer sentido sob condição (por exemplo, saldo suficiente).
- Use `@settings(max_examples=30)` na propriedade que envolva múltiplas requisições HTTP ao servidor.
- Use `float()` para garantir tipo compatível com JSON quando aplicável.
- Exclua NaN e Infinity das estratégias de floats (`allow_nan=False`, `allow_infinity=False`).
- Importe `requests` para fazer as requisições HTTP ao servidor Flask local.

Gere apenas o código Python. Sem explicações adicionais.

Prompt completo para o GitHub Copilot - geração do `test_hypothesis.py`

Uso: abra o GitHub Copilot Chat no VSCode (Ctrl+Alt+I no Windows / Cmd+Shift+I no macOS) com o arquivo `test_hypothesis.py` criado e vazio no editor. Copie todo o bloco abaixo e cole no campo de chat do Copilot. Pressione Enter. O Copilot deve gerar o código Python pronto para o pytest.

Gere o conteúdo de um arquivo `test_hypothesis.py` para testes baseados em propriedades com a biblioteca `Hypothesis` em Python, aplicados ao sistema de internet banking da disciplina de Teste de Software.

Servidor Flask local em `http://127.0.0.1:5000` com os seguintes endpoints:

- GET `/saldo/<conta_id>`
- POST `/transferencia` — corpo JSON: `{"origem":int,"destino":int,"valor":float}`
- GET `/extrato/<conta_id>`

Estado inicial das contas (mantido pelo conftest.py existente na mesma pasta, com fixture autouse): id=1 saldo 1000.00; id=2 saldo 500.00; id=3 saldo 0.00.

Invariantes a serem implementadas como propriedades:

Invariante 1: para qualquer valor não-positivo (zero ou negativo) submetido ao endpoint /transferencia, o sistema deve sempre retornar HTTP 422 - valor inválido.

Invariante 2: para qualquer ID inteiro que não pertença ao conjunto de contas cadastradas no sistema (ou seja, ID fora do conjunto {1, 2, 3}), o endpoint /saldo/<conta_id> deve sempre retornar HTTP 404 - conta não encontrada.

Invariante 3: para qualquer transferência aceita pelo endpoint /transferencia (resposta HTTP 200), a soma dos saldos das contas de origem e destino imediatamente após a operação deve ser exatamente igual à soma dos saldos imediatamente antes da operação - $\text{saldo}(\text{origem}, \text{antes}) + \text{saldo}(\text{destino}, \text{antes}) = \text{saldo}(\text{origem}, \text{depois}) + \text{saldo}(\text{destino}, \text{depois})$.

Requisitos obrigatórios:

- Use @given e estratégias strategies as st da biblioteca hypothesis para gerar inputs automaticamente.
- Use assume(...) quando a propriedade só fizer sentido sob condição (por exemplo, saldo suficiente).
- Use @settings(max_examples=30) na propriedade que envolva múltiplas requisições HTTP ao servidor.
- Use float() para garantir tipo compatível com JSON quando aplicável.
- Exclua NaN e Infinity das estratégias de floats (allow_nan=False, allow_infinity=False).
- Importe requests para fazer as requisições HTTP ao servidor Flask local.

Gere apenas o código Python. Sem explicações adicionais.

Passo 4. Com o servidor Flask ainda rodando no Terminal 1, execute o arquivo gerado no Terminal 2:

```
pytest test_hypothesis.py -v
```

Investigação dos contraexemplos: se o Hypothesis encontrar um contraexemplo, ele exibirá no terminal o menor valor (após *shrinking*) que viola a propriedade. Para cada contraexemplo encontrado, registre em seu arquivo Word:

- a) qual foi o valor mínimo encontrado pelo shrinking;

b) o sistema estava errado (defeito real - categoria (C) do Sprint 2) ou a propriedade estava mal formulada (categoria (A))?

c) o que esse contraexemplo revela sobre o domínio bancário que nem a sua Q8 nem o test_ia_gerado.py do Sprint 1 haviam considerado?

Resultado esperado:

```
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
• MacBook-Pro-de-Claudinei:internet_banking_tests claudinei$ source /Users/internet_banking_tests/venv/bin/activate
• (venv) MacBook-Pro-de-Claudinei:internet_banking_tests claudinei$ pytest test_hypothesis.py -v
===== test session starts =====
platform darwin -- Python 3.13.2, pytest-9.0.3, pluggy-1.6.0 -- /Users/internet_banking_tests/venv/bin/python3.13
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /Users/internet_banking_tests
plugins: hypothesis-6.152.7, cov-7.1.0, bdd-8.1.0
collected 3 items

test_hypothesis.py::test_invariante_1_valor_nao_positivo_sempre_422 PASSED [ 33%]
test_hypothesis.py::test_invariante_2_conta_inexistente_sempre_404 PASSED [ 66%]
test_hypothesis.py::test_invariante_3_conservacao_de_valor PASSED [100%]

===== 3 passed in 23.44s =====
(venv) MacBook-Pro-de-Claudinei:internet_banking_tests claudinei$
```

Passo 5. Preencha a tabela abaixo comparando o que a IA gerou no Sprint 1 com o que o Hypothesis encontrou neste sprint. As cinco linhas representam casos críticos do domínio bancário:

Caso	Gerado pela IA?	Encontrado pelo Hypothesis?	Relevância bancária
Valor 0.0 rejeitado			
Valor -0.001 rejeitado			
Conta ID 9999 → 404			
Saldo conservado em múltiplas transferências			
Float com muitas casas decimais			

Resultado esperado:

Consolidado da execução: três invariantes verificadas pelo Hypothesis com status PASSED em 23,44 segundos. A tabela abaixo confronta o que a IA cobriu no Sprint 1 (testes baseados em exemplos pontuais) com o que o Hypothesis efetivamente explorou neste sprint (testes baseados em propriedades, com inputs gerados automaticamente).

Caso	Gerado pela IA?	Encontrado pelo Hypothesis?	Relevância bancária
Valor rejeitado 0.0	Sim test_transferencia_valor_zero	Sim Invariante 1 PASSED	Impede operações nulas no extrato e preserva a integridade contábil, uma transferência de R\$ 0,00 não tem significado financeiro e poluiria o histórico do cliente.
Valor rejeitado -0.001	Parcial . a IA testou apenas valor = -50; não cobriu bordas próximas a zero.	Sim , Invariante 1 PASSED, espaço amostral inclui valores negativos arbitrariamente pequenos.	Valores negativos próximos a zero podem escapar de validações superficiais do tipo if valor < -1; risco de “transferência reversa” que retira saldo da conta de destino em vez de creditar.
Conta ID 9999 → 404	Parcial , a IA testou um único ID específico de conta inexistente, não o conjunto universal de IDs fora de {1,2,3}.	Sim , Invariante 2 PASSED, com IDs gerados entre -9999 e 9999.	Coerência universal da política de erro: qualquer ID inválido recebe o mesmo tratamento (404). Tratamentos diferenciados poderiam expor o range de IDs válidos a tentativas de enumeração, vetor clássico de reconhecimento em ataques.
Saldo conservado em múltiplas transferências	Não , a IA verificou saldo individual após transferência, nunca a soma conservada entre as duas contas.	Sim , Invariante 3 PASSED, com 30 transferências de valores variados.	Princípio fundamental de qualquer sistema financeiro. Violação significaria criar ou destruir dinheiro por defeito de software — equivalente a fraude algorítmica, com impacto

Caso	Gerado pela IA?	Encontrado pelo Hypothesis?	Relevância bancária
			regulatório imediato em ambiente real.
Float com muitas casas decimais	Não, a IA usou valores limpos como 100, 200, 1000 e -50.	Sim, Invariante 3 PASSED, com floats gerados em amplo espectro de precisão decimal.	Erros de ponto flutuante (IEEE 754) podem causar inconsistências em centavos ao longo do tempo, por exemplo, $0.1 + 0.2 \neq 0.3$ exatamente. Sistemas bancários em produção exigem aritmética decimal (Decimal em Python) ou armazenamento em centavos como inteiros.

Leitura da tabela: em três das cinco linhas (Valor -0.001, Conta ID 9999, Saldo conservado em múltiplas transferências e Float com muitas casas decimais) o Hypothesis *complementou* a cobertura da IA, explorando regiões do espaço de inputs que a especificação humana da Q8 não havia descrito. Em apenas uma linha (Valor 0.0) houve sobreposição plena. O resultado PASSED em todas as três invariantes não comprova que o sistema está livre de defeitos - comprova que, para os centenas de inputs gerados pelo Hypothesis no espaço delimitado pelas estratégias, nenhum contraexemplo foi encontrado.

Reflexão do Sprint 3 - registre em seu arquivo Word antes de avançar

Responda com suas próprias palavras. Não existe resposta certa. O que importa é que o texto revele o seu raciocínio real durante a execução.

1. O Hypothesis encontrou algum caso que a IA não gerou no Sprint 1? Se sim, qual é a relevância desse caso para um sistema bancário real em produção?

2. Qual é a diferença fundamental entre “gerar exemplos” e “verificar propriedades”? Use suas próprias palavras - sem copiar a definição do texto.

3. Depois de executar o Hypothesis, você confia mais ou menos nos testes gerados pela IA nos Sprints 1 e 2? Justifique com base no que você observou empiricamente, citando MacIver et al. (2019).

4. Existe alguma invariante do internet banking que o Hypothesis não conseguiria verificar automaticamente? Qual e por quê?

Atividade complementar - formule duas novas invariantes com Hypothesis

Objetivo: no Sprint 3 trabalhamos com três invariantes do domínio do internet banking, formuladas a partir de pistas do roteiro. Nesta atividade complementar você opera com autonomia plena: formula duas invariantes inéditas a partir das regiões do sistema que as três originais não exploraram, gera o código com o GitHub Copilot e verifica empiricamente se o sistema as respeita.

Pré-requisito: o arquivo `test_hypothesis.py` original (disponibilizado no AVA pelo professor) deve estar na pasta `internet_banking_tests` junto com o `app.py`, o `conftest.py` e o `venv` ativado.

Passo 1. Releia o `test_hypothesis.py` original e identifique **regiões do sistema de internet banking que ele não testa**. A lista abaixo serve como referência de áreas inexploradas, mas você pode propor outras:

Regiões inexploradas pelas três invariantes originais:

- o endpoint `/extrato/<conta_id>` - nunca foi alvo de propriedade nas três invariantes originais;
- transferência com origem = destino - qual deve ser o comportamento universal do sistema?
- transferência com *valor estritamente maior que o saldo* (saldo insuficiente) — propriedade universal de rejeição;
- *relação entre saldo antes e depois* de uma transferência aceita (para a conta individual de origem ou destino);
- *rastreabilidade no extrato* - após qualquer transferência aceita, o extrato deve registrar a operação;

- *robustez de tipo* - valor não-numérico (string) no campo valor do JSON.

Passo 2. Em seu arquivo Word, formule **duas invariantes inéditas** no formato

Invariante N: para qualquer X em domínio D, a propriedade P se verifica.

Critérios obrigatórios para que a formulação seja aceita como invariante (e não como caso pontual):

- a) deve aplicar-se a **qualquer** valor dentro de um domínio bem definido, não a um valor específico;
- (b) deve ser *implementável com estratégias do Hypothesis* (st.floats, st.integers, st.text, st.tuples, etc.);
- (c) deve ser *verificável por requisição HTTP* a um dos três endpoints do sistema;
- (d) **não pode duplicar** o escopo de nenhuma das três invariantes originais, precisa cobrir região nova.

Passo 3. Inicie o servidor Flask no Terminal 1 (python app.py), abra o GitHub Copilot Chat no VSCode (Ctrl+Alt+I / Cmd+Shift+I), crie no editor um **novo arquivo** chamado test_hypothesis_extra.py, não sobrescreva o test_hypothesis.py original. Cole o prompt abaixo no chat, substituindo [SUAS_DUAS_INVARIANTES] pelas duas formulações do Passo 2, e pressione Enter:

Gere o conteúdo de um arquivo test_hypothesis_extra.py para testes baseados em propriedades com a biblioteca Hypothesis em Python, aplicados ao sistema de internet banking da disciplina de Teste de Software.

Servidor Flask local em http://127.0.0.1:5000 com endpoints: GET /saldo/<conta_id>; POST /transferencia (corpo JSON: {"origem":int,"destino":int,"valor":float}); GET /extrato/<conta_id>. Estado inicial mantido pelo confest.py com fixture autouse: id=1 saldo 1000.00; id=2 saldo 500.00; id=3 saldo 0.00.

Invariantes a serem implementadas como propriedades:

[SUAS_DUAS_INVARIANTES]

Requisitos obrigatórios: use @given e strategies as st; use assume(...) quando a propriedade só fizer sentido sob condição; use @settings(max_examples=30) em propriedades com múltiplas requisições HTTP; use float() para garantir tipo compatível

com JSON quando aplicável; exclua NaN e Infinity de estratégias de floats (allow_nan=False, allow_infinity=False); importe requests. Gere apenas o código Python. Sem explicações adicionais.

Passo 4. Aplique o código gerado no arquivo, salve, abra o Terminal 2 e execute:

```
pytest test_hypothesis_extra.py -v
```

Passo 5. Para *cada* uma das duas invariantes, registre em seu arquivo Word:

- a) o enunciado da invariante na forma “para qualquer X em D, vale P”;
- b) o resultado do pytest - PASSED ou Falsifying example com o menor valor encontrado pelo shrinking;
- c) se houve falha, a classificação na taxonomia A/B/C/D do Sprint 2 (erro de teste, erro de API, defeito real, erro de isolamento) e a justificativa;
- d) a relevância da invariante no domínio bancário - qual consequência operacional sua violação teria em produção?

Entrega no AVA junto com os sprint 1 e 2 : arquivo Word único contendo, na ordem: (1) as duas invariantes formuladas; (2) o código completo do test_hypothesis_extra.py gerado pelo Copilot, copiado e colado; (3) o output do pytest copiado do Terminal 2; (4) a análise crítica das duas invariantes conforme o Passo 5 (tabela preenchida). Vale até 8 pontos. A nota considera a originalidade das invariantes (não-duplicação das três originais), a aderência ao formato universal, a coerência entre formulação e código gerado e a profundidade da análise crítica e sua presença no texto.

Poste aqui o resultados dos 3 *sprint* reformulados – Tabelas e reflexões – em um único arquivo – vale até 10 pontos.

Referências bibliográficas

BSTQB — BRAZILIAN SOFTWARE TESTING QUALIFICATIONS BOARD. *Certified Tester Foundation Level Syllabus*: versão 4.0. [S. l.]: ISTQB; BSTQB, 2023. Capítulo 6, p. 52-53. Disponível em: https://www.bstqb.online/files/syllabus_ctfl_4.0br.pdf. Acesso em: maio 2026.

MacIVER, D. R.; HATFIELD-DODDS, Z. et al. Hypothesis: a new approach to property-based testing. *Journal of Open Source Software*, v. 4, n. 43, p. 1891, 2019. DOI: 10.21105/joss.01891. Disponível em: <https://joss.theoj.org/papers/10.21105/joss.01891>. Acesso em: maio 2026.

COUTINHO, N. de M.; NASCIMENTO, E. L. B. do. Desafios e benefícios da implementação de testes automatizados em empresas de software. *Cuadernos de Educación y Desarrollo*, v. 17, n. 4, e8221, p. 1-19, 2025. DOI: <https://doi.org/10.55905/cuadv17n4-176>. Disponível em: <https://ojs.cuadernoseducacion.com/ojs/index.php/ced/article/view/8221>. Acesso em: maio 2026.